

Philipps



Universität  
Marburg

# Distributed Incremental Streaming based Inclusion Dependency Discovery

*Expose of Master Thesis*

Ram Binay Gupta

Supervisors:

Prof. Dr. Thorsten Papenbrock

April 15, 2026

**Abstract** Inclusion Dependencies (INDs) are essential constraints in relational data, widely used in data integration, schema matching, and data cleaning. However, existing approaches either assume static datasets or suffer from high memory and computational costs when applied to dynamic environments. This work proposes a scalable and memory efficient approach for incremental IND discovery in distributed systems using the Akka actor model which has rarely been explored. The approach combines exact tracking for possible valid unary INDs with approximate and cheap strategies for efficient pruning of irrelevant INDs. This hybrid design significantly reduces memory requirements while maintaining exactness for relevant INDs. The system aims to achieve state of the art (SOT) performance for large, continuously evolving datasets.

## 1 Introduction

Inclusion Dependencies (INDs) express containment relationships between attributes, such as  $A \subseteq B$ , indicating that all values in column  $A$  appear in column  $B$ . These dependencies are fundamental for tasks such as data integration [9], schema matching [8], and referential integrity validation [4].

Traditional IND discovery algorithms are designed for static datasets and require full recomputation when new data arrives. With modern applications generating continuous data streams, such approaches become inefficient and impractical [13].

Although incremental and distributed IND discovery have been investigated independently, their combination remains largely unexplored due to the high complexity involved. Existing incremental systems often store value to column mappings, leading to memory complexity of  $O(|V| \times |A|)$ , where  $|V|$  is the number of distinct values and  $|A|$  the number of attributes. The paper aims to answer the following research questions.

1. RQ1: How can the system compute incremental INDs over a distributed environment?
2. RQ2: How can the system be optimized for memory and performance?

## 2 Related Work

A variety of approaches have been employed for the discovery of unary inclusion dependencies (INDs), primarily in static batch oriented settings. Early work by Bell and Brockhausen introduces pruning strategies based on logical inference and column statistics, such as data types and value ranges, to reduce the search space [3]. While effective in limiting candidate checks, their approach relies on SQL join operations, requiring full

data loading and incurring significant query overhead. Such assumptions have limited applicability in streaming scenarios, where data is continuously evolving and schema characteristics may not be known in advance.

De Marchi et al. improve candidate validation efficiency by employing an inverted index that maps each value to the set of attributes containing it [5]. Inclusion dependencies can then be identified through intersections of these attribute sets. However, this approach suffers from scalability limitations, as the inverted index can grow very large, leading to increased memory consumption and costly disk I/O when spilling occurs.

To address these limitations, Bauckmann et al. propose SPIDER [2], a sort merge based algorithm that reduces memory requirements by operating on sorted attribute files stored on disk. This approach avoids maintaining large in memory indices. They also explore the use of Bloom filters to prune candidates, though the resulting performance improvements remain limited due to numerous sorting and disk I/O operations.

Distributed processing has been explored with SINDY [7], which reformulates IND discovery as a MapReduce style pipeline. Similarly, incremental discovery has been studied by Shaabani and Meinel through the S-INDD [14] family of algorithms. These methods maintain clusters of attributes based on shared values, thereby reducing the need for explicit candidate validation. Although they support updates and deletions, their reliance on attribute clustering introduces additional data structures and maintenance overhead. In contrast, our approach avoids global clustering and instead relies on value centric state and local updates.

Recent work by Müller et al. introduces SPIND [10], a scalable algorithm for discovering both exact and partial inclusion dependencies (pINDs), that unifies both within a single framework. The algorithm employs a chunking based global sortation strategy that compresses multiple column sortations into a small number of files, significantly reducing I/O overhead. Furthermore, SPIND [10] introduces a violation aware validation scheme that counts inclusion violations instead of immediately pruning candidates, enabling efficient discovery of partial dependencies. Its level wise candidate generation, combined with probabilistic filtering and parallel processing, allows SPIND [10] to scale effectively to large datasets and higher arity dependencies. Empirical results show that SPIND [10] outperforms existing state-of-the-art algorithms by orders of magnitude in certain scenarios, particularly for n-ary IND discovery.

Despite these advances, most existing approaches including SPIND [10] are designed for static or batch processing environments. They rely on global sorting, indexing, or repeated scans over the dataset, which limits their applicability in continuously evolving data settings. Moreover, many techniques depend on global data structures (e.g., inverted indices, attribute clusters, or sorted files), which can become bottlenecks in distributed or memory constrained environments.

These limitations motivate the need for a scalable, incremental, and distributed IND discovery approach that avoids global structures, minimizes memory overhead, and supports efficient updates under continuous data evolution.

### 3 Preliminaries and Definitions

Let  $r$  be a relational dataset with schema  $R$ .

**Definition 1 (Inclusion Dependency).** An IND  $A \subseteq B$  holds if:

$$\forall v \in V(A), v \in V(B)$$

where  $V(A)$  denotes the set of distinct values in column  $A$ .

**Definition 2 (Violation Set).** For a candidate IND  $A \subseteq B$ , the violation set is:

$$\text{excl}(A, B) = V(A) \setminus V(B)$$

The IND holds if and only if:

$$|\text{excl}(A, B)| = |\emptyset| = 0$$

**Definition 3 (Incremental Update).** Given new data  $\Delta r$ , the dataset becomes:

$$r \leftarrow r \cup \Delta r$$

The goal is to update INDs efficiently without recomputing from scratch.

### 4 Approach

This paper proposes a hybrid layered architecture with value centric and event driven design. Each distinct value is associated with its current attribute membership, which captures the set of attributes in which the value appears alongwith the count. Updates to the dataset are processed as events that modify this membership.

The IND validity depends only on whether a value belongs to an attribute, not on its frequency. Therefore, only transitions in value membership when a value enters or leaves an attribute are relevant for triggering further computations. These transitions are captured as events that are propagated to the maintenance layer.

To ensure scalability, the paper proposes not to maintain all possible attribute pairs. Instead, it uses approximate summaries to identify promising candidate pairs and maintains exact violation counters only for these active pairs. This selective strategy avoids the explosion of the search space while preserving correctness for tracked dependencies. The tracking of IND pairs for exactness or approximate is automatically decided by the algorithm.

The paper proposes a four staged pipeline : value maintenance, change propagation, candidate validation and candidate selection.

## 4.1 Value Maintenance

Every incoming updates like  $\text{INSERT}(A, v)$ ,  $\text{DELETE}(A, v)$  are decomposed into attribute value events. For each value, the system maintains its current attribute membership. Whenever a value is inserted into or removed from an attribute, its membership is updated accordingly. An event is generated only when this update results in a change of membership,  $0 \rightarrow 1$  (enter),  $1 \rightarrow 0$  (leave) i.e. when a value appears in an attribute for the first time or disappears completely from it.

## 4.2 Change Propagation

Each event describes a change in the membership of a value with respect to a specific attribute. These events are propagated to IND candidate managers responsible for maintaining dependency state. The propagation is restricted to candidate pairs that involve the affected attribute, ensuring that updates remain localized and efficient.

## 4.3 Candidate Validation

For each active pair of attributes  $(A, B)$ , the system maintains a violation or exclusion counter. When a value enters attribute  $A$ , the counter for  $(A, B)$  is incremented if the value does not belong to  $B$ . Conversely, when a value enters  $B$ , the counter for  $(A, B)$  may decrease if the value previously contributed to a violation. Similar updates apply when values leave attributes. Through these localized updates, the system maintains exact violation counts incrementally.

## 4.4 Candidate Selection

To limit the number of tracked pairs, the paper proposes approximate summaries for candidate selection. Lightweight structures such as the Counting Quotient Filter[11], HyperLogLog [6] sketches and maintaining top-k frequent values per columns are used to quickly discard pairs that are unlikely to satisfy containment. Only pairs that pass these filters are promoted to active tracking. Upon promotion, an initialization step computes the initial violation count to ensure correctness. Similarly, when active pairs are no more likely to be INDs, they are demoted and lose their active tracking.

## 4.5 High Level Algorithm

---

**Algorithm 1** Distributed Incremental Streaming - IND

---

```
1: for each batch of updates do
2:   Update value memberships
3:   Generate change events
4:   Propagate events to IND candidate managers
5:   Update violation counters for active pairs
6:   if Promotion condition is met then
7:     Select and Activate candidate pairs
8:     output "Valid IND pairs"
9:   end if
10:  if Demotion condition is met then
11:    Deactivate unpromising pairs
12:  end if
13: end for
```

---

## 4.6 Results

The algorithm periodically outputs the valid IND pairs whose violation counter is zero i.e  $|\text{excl}(A, B)| = |\emptyset| = 0$ . The paper expects significant reduction in memory usage, efficient incremental updates and accurate IND detection for relevant pairs.

## 5 Evaluation

The system will be evaluated on real world datasets with high cardinality and dynamic updates. The paper proposes to compare the efficiency with the State-of-the-art algorithm such as SPIND [10], BINDER [12] and the approach mentioned by Shaabani et al. [13] over benchmark datasets such as TESMA [1], TPCB [15]. The primary metrics for comparisons include runtime performance, memory consumption and accuracy of detected INDs. Scalability will be evaluated by increasing dataset size and attribute dimensionality under parallel execution.

## References

- [1] B. Alexe, W.-C. Tan, and Y. Velegrakis. Stbenchmark: Towards a benchmark for mapping systems. In *Proceedings of the VLDB Endowment (PVLDB)*, volume 1, pages 230–244, 2008.
- [2] J. Bauckmann, U. Leser, F. Naumann, and V. Tietz. Efficiently detecting inclusion dependencies. In *IEEE International Conference on Data Engineering*, pages 1448–1450, 2007.

- [3] S. Bell and P. Brockhausen. Discovery of data dependencies in relational databases. In *Proceedings of the Conference on Data Engineering*, 1995.
- [4] F. De Marchi, S. Lopes, and J.-M. Petit. Efficient algorithms for mining inclusion dependencies. In *International Conference on Extending Database Technology*, pages 464–476. Springer, 2002.
- [5] F. De Marchi, S. Lopes, and J.-M. Petit. Unary and n-ary inclusion dependency discovery in relational databases. *Journal of Intelligent Information Systems*, 32(1):53–73, 2009.
- [6] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier. Hyperloglog: The analysis of a near-optimal cardinality estimation algorithm. In *Proceedings of the 2007 International Conference on Analysis of Algorithms (AofA)*, pages 127–146. DMTCS, 2007. INRIA research report RR-2007-024.
- [7] S. Kruse, T. Papenbrock, and F. Naumann. Scaling out the discovery of inclusion dependencies. In *BTW Conference*, pages 445–454, 2015.
- [8] M. Levene and M. W. Vincent. Justification for inclusion dependency normal form. *IEEE Transactions on Knowledge and Data Engineering*, 12(2):281–291, 2000. doi: 10.1109/69.842271.
- [9] R. J. Miller, M. A. Hernández, L. M. Haas, L.-L. Yan, C. T. H. Ho, R. Fagin, and L. Popa. The clio project: Managing heterogeneity. *SIGMOD Record*, 30(1):78–83, 2001. doi: 10.1145/373626.373693.
- [10] J. L. Müller, M. Seeger, and T. Papenbrock. Spind: Scalable partial inclusion dependency discovery. In *EDBT*, 2026. OpenProceedings.org, ISBN 978-3-98318-104-9.
- [11] P. Pandey, M. A. Bender, R. Johnson, and R. Patro. A general-purpose counting filter: Making every bit count. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*, pages 775–787. ACM, 2017. doi: 10.1145/3035918.3064057.
- [12] T. Papenbrock, S. Kruse, J.-A. Quiané-Ruiz, and F. Naumann. Divide & conquer-based inclusion dependency discovery. *Proceedings of the VLDB Endowment*, 8(7): 774–785, 2015.
- [13] N. Shaabani and C. Meinel. Incremental discovery of inclusion dependencies. Available at: nuhad.shaabani@hpi.de.
- [14] N. Shaabani and C. Meinel. Scalable inclusion dependency discovery. In *DASFAA*, pages 425–440. Springer, 2015.
- [15] Transaction Processing Performance Council. Tpc benchmark h (decision support) standard specification. <http://www.tpc.org/tpch/>, 2014. Version 2.17.3 (or latest available).